Conceptual design document: Grocery shop inventory management system

Detailed Technical Design — Hybrid C11 and C++17

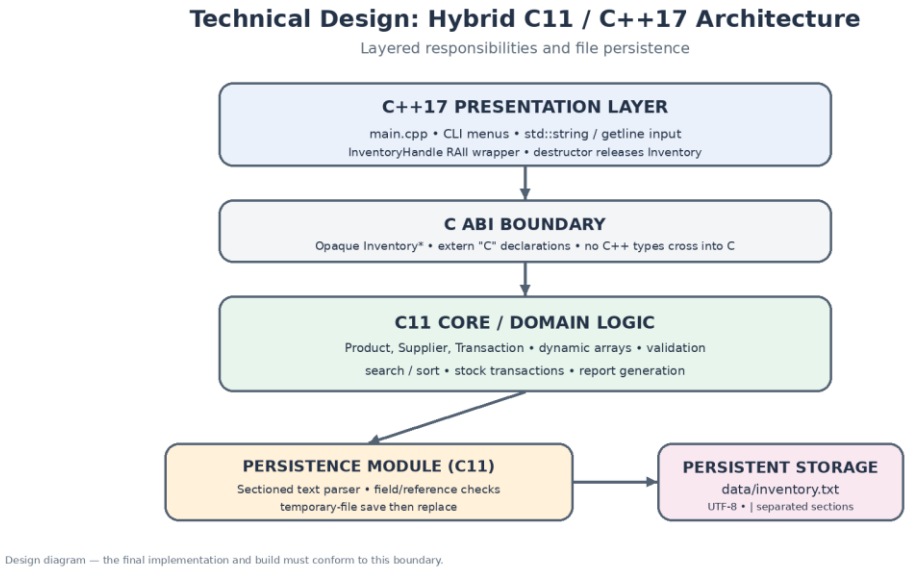
1. Purpose and domain

This document describes the planned technical design for a command-line inventory management system for a small grocery shop. It manages products, suppliers, stock transactions, reports, and persistent data. The implementation uses a C11 core for domain logic and a small C++17 frontend for the command-line interface and resource management. Baseline categories include cooking oil, rice, sugar, flour, canned foods, drinks, and other. Each product has a fixed unit. Stock must not become negative, and a product with transaction history cannot be deleted. Expiry-date tracking and a graphical user interface are outside the baseline scope.

2. Architecture and language responsibilities

The system separates presentation, language interoperability, core logic, and persistence. The C++17 layer collects input and displays menus and reports. The C11 core owns data structures, validates operations, updates stock, searches and sorts records, generates reports, and reads/writes files.

Layer	Responsibilities
C11 core	Entities, dynamic arrays, validation, stock operations, search/sort, persistence, reports.
C++17 frontend	CLI menu, std::string/getline input, presentation and RAII wrapper.
C/C++ boundary	Opaque Inventory* and extern "C" declarations; no C++ types or exceptions cross into C.
Persistence	Sectioned UTF-8 text file, parsing and validation, safe save through temporary file replacement.



Design diagram — the final implementation and build must conform to this boundary.

3. Expected system design and planned steps

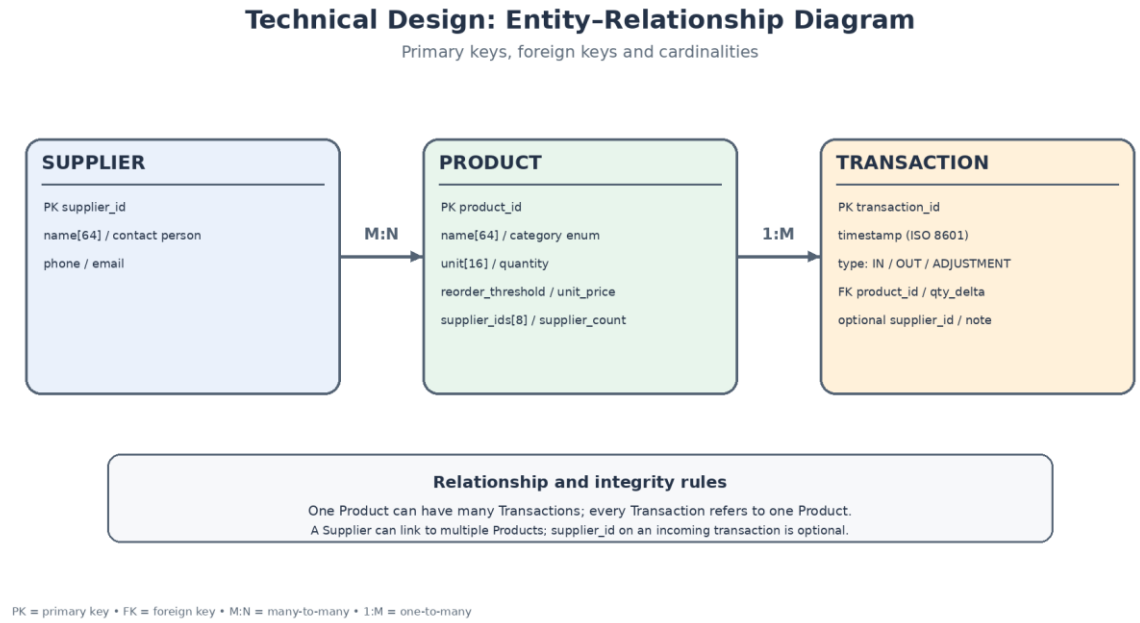
The C++17 CLI collects choices and displays results. It calls an RAII wrapper, which owns the opaque inventory handle and releases it through inventory_destroy(). The wrapper calls functions exposed by the C API. The C11 core validates requests, updates records, performs searches and reports, and uses the persistence component to load or save data.

Component	Responsibility	Planned files / interface
C++17 CLI	Menu, prompts, input and output.	src/main.cpp
C++ RAII wrapper	Owns handle and ensures cleanup.	src/inventory_handle.hpp
C API boundary	C-compatible declarations for C++ callers.	include/inventory.h and related headers; extern "C" guards
C11 core	Entities, validation, stock operations, search, sorting, reports.	src/inventory.c and supporting modules
Persistence	Parse, validate, load and save records.	src/persistence.c / include/persistence.h
Data file	Persistent records.	data/inventory.txt

These are planned responsibilities. Match exact filenames and signatures to the supplied skeleton; retain the separation between interface, core logic and persistence.

4. Entity model and relationships

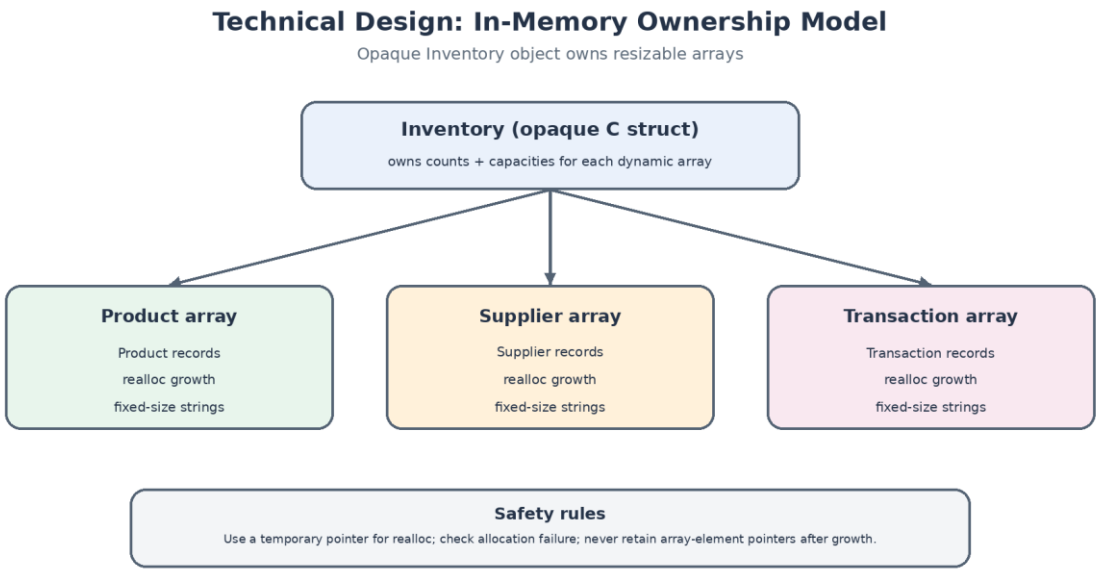
The design has three primary entities. Products and suppliers are associated many-to-many because a supplier may provide several products and a product may be sourced from several suppliers. Each transaction refers to one product, while a product may have many transactions. An incoming transaction may optionally record a supplier.



Entity	Main fields	Relationship notes
Product	product_id, name, category, unit, quantity, reorder threshold, unit price, supplier IDs	One product has many transactions; can link to multiple suppliers.
Supplier	supplier_id, name, contact person, phone, email	Can supply multiple products; optional on incoming transactions.
Transaction	transaction_id, timestamp, type, product_id, quantity delta, optional supplier_id, note	Each transaction refers to one product.

5. In- memory data structures and ownership

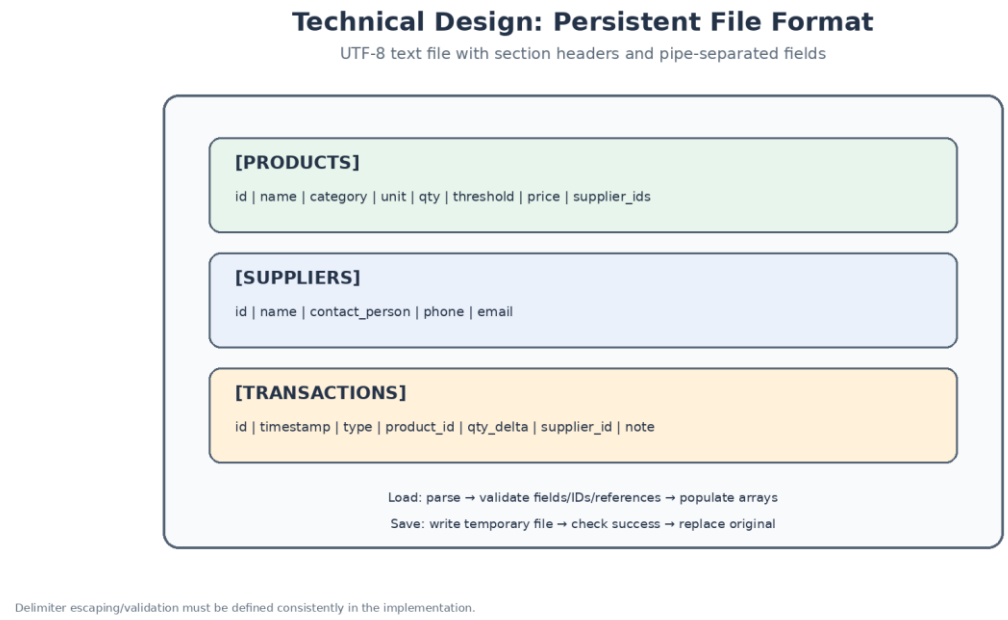
An opaque Inventory structure owns dynamic arrays for products, suppliers, and transactions. Each array tracks its count and capacity and grows as needed using realloc, typically doubling capacity. Fixed-size character arrays hold short text fields. Lookup can use linear scans, while sorting can use qsort (uses function pointers for comparators).



Memory-safety rules: store realloc results in a temporary pointer and check allocation failure; release owned memory in inventory_destroy(); and do not retain pointers to array elements across operations that may reallocate an array. The C++ wrapper uses RAII so the C inventory is destroyed when the wrapper leaves scope.

6. Persistence and file format

The proposed persistent file is data/inventory.txt, encoded as UTF-8 with section headers and pipe-separated fields. The loader should validate record fields, unique IDs, enum values, numeric values, and foreign-key references. A missing file should result in an empty inventory; malformed records should be reported clearly.



Section	Fields (separated by |)
[PRODUCTS]	id | name | category | unit | qty | threshold | price | supplier_ids
[SUPPLIERS]	id | name | contact_person | phone | email
[TRANSACTIONS]	id | timestamp | type | product_id | qty_delta | supplier_id | note

Saving should write a complete new version to a temporary file, verify successful writes and close, then replace the original file. The implementation must consistently define whether pipe characters in fields are escaped or rejected. Pipe characters (\|) are rejected in all text fields during input validation to prevent file corruption and preserve the pipe-separated format

7. Operations and CLI menu

Planned command-line menu:

- 1. Products — add, edit, remove, search, list, and sort
- 2. Suppliers — add, edit, remove, and list suppliers
- 3. Transactions — stock in, stock out, and stock adjustment
- 4. Reports — current stock, low-stock items, transaction history, and inventory value
- 5. Save inventory
- 6. Load inventory
- 0. Exit

Stock-in increases quantity; stock-out decreases quantity only when enough stock exists; adjustments record a correction. Reject invalid quantities, unknown product IDs, duplicate IDs, and operations that violate domain rules.

8. Toolchain, build and testing

Use a C11-capable GCC or Clang compiler for .c files and a C++17-capable compiler for .cpp files. A Makefile should compile the languages separately and perform final linking with the C++ compiler. C headers should wrap declarations in #ifdef __cplusplus / extern "C" guards

Test area	Expected verification
CRUD and search	Add/edit/remove/find products and suppliers; check duplicate IDs and protected deletion.
Stock transactions	Stock-in/out/adjustments update quantities and stock never becomes negative.
Persistence	Save, restart and reload; compare loaded records with saved records.
Invalid data	Malformed lines, invalid enums/numbers, missing references, and truncated records are handled.
Memory safety	Exercise array growth and cleanup; use Valgrind or equivalent where available.
Reports	Current stock, low stock, transaction history and inventory value match test fixtures.

9. Repository structure

- include/ — public C headers
- src/ — C core modules, C++ wrapper header, and main.cpp
- data/ — persistent inventory data (avoid committing sensitive real data)
- docs/ — design document and diagrams
- Makefile — build targets
- README.md — setup, build, run, tests, and known limitations

10. Scope and possible extensions

Baseline: command-line interface, product/supplier records, stock transactions, search/sort, reports, validation, and persistent storage. Possible extensions include undo, indexed lookup, import/export, expiry-date tracking, and a graphical interface.

11. References

1. Hunt, A., & Thomas, D. (2019). The Pragmatic Programmer (20th Anniversary ed.). Addison-Wesley.
2. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
3. ISO/IEC 9899:2018. Programming Languages — C.
4. ISO/IEC 14882:2017. Programming Languages — C++.